
[Description](#)

[Intended User](#)

[Features](#)

[User Interface Mocks](#)

[Screen 1](#)

[Screen 2](#)

[Key Considerations](#)

[How will your app handle data persistence?](#)

[Describe any corner cases in the UX.](#)

[Describe any libraries you'll be using and share your reasoning for including them.](#)

[Describe how you will implement Google Play Services.](#)

[Next Steps: Required Tasks](#)

[Task 1: Project Setup](#)

[Task 2: Implement UI for Each Activity and Fragment](#)

[Task 3: Your Next Task](#)

[Task 4: Your Next Task](#)

[Task 5: Your Next Task](#)

GitHub Username: atmko

On My Watch

Description

On my watch is designed to display and search for recent, popular and upcoming movies and television shows.

Create your own lists of movies and shows. Never forget what you've watched and never forget what you want to continue watching.

Set notifications to know about dvd / blu-ray releases and releases to online streaming networks such as netflix.

Intended User

Anyone who would like to track and share their viewing history.

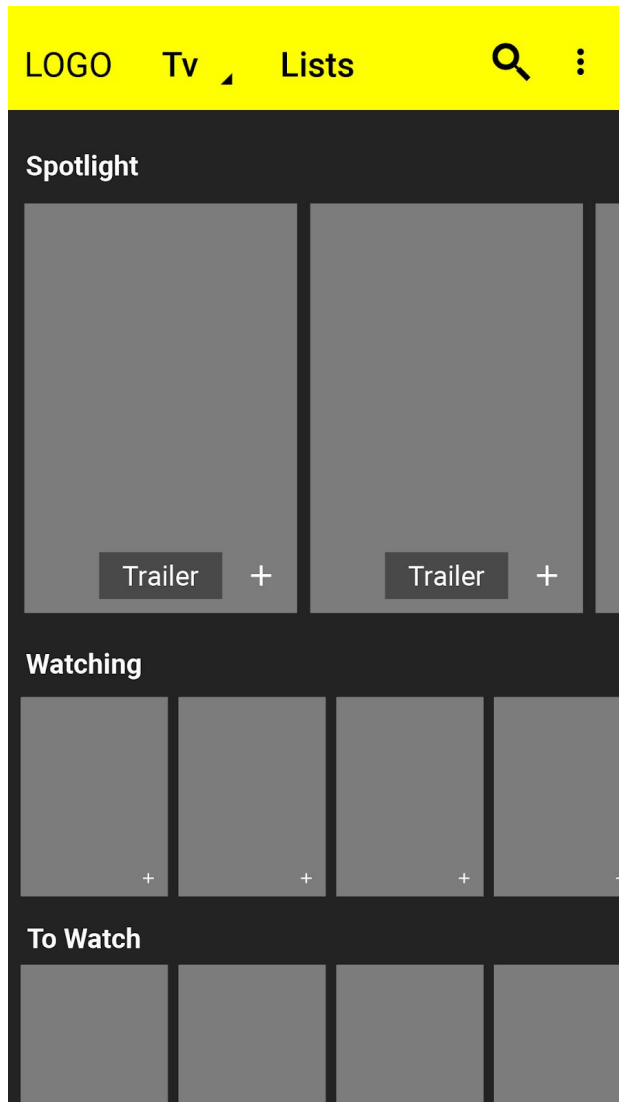
Anyone who watches a large amount of content and would like to be reminded of which shows that they are following.

Features

- Create your own lists of movies and shows
- Receive notifications on new show episodes / movie releases for items in your lists.
- Set notifications to track releases on DVD, Blu-ray and online streaming networks such as netflix.
- Rate movies and shows
- Backup to Google Drive

User Interface Mocks

Home Screen



This is the first screen shown when the app is launched.

It features navigation to others screens (settings, lists and search).

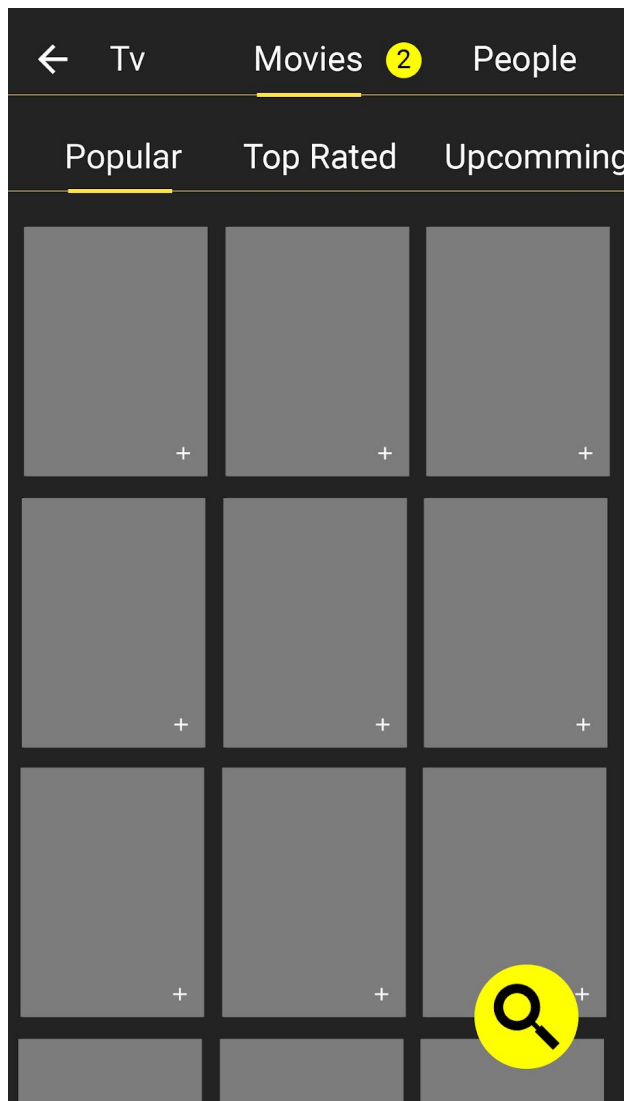
The screen's primary function is to show at a glance information that would be of interest to the user.

The “Spotlight” section displays media that is widely popular locally and internationally.

The subsequent sections display items in lists that the user has added and can be quickly reminded about. This includes new episodes of shows that the user is currently watching.

The app bar has a spinner toggle that aids the user in switching from the tv show home screen and the movies homescreen.

Search Screen (Preset mode)



This screen helps the user to find new media to add to their lists. The top section is divided into panel that enables easy navigation of either movies or shows.

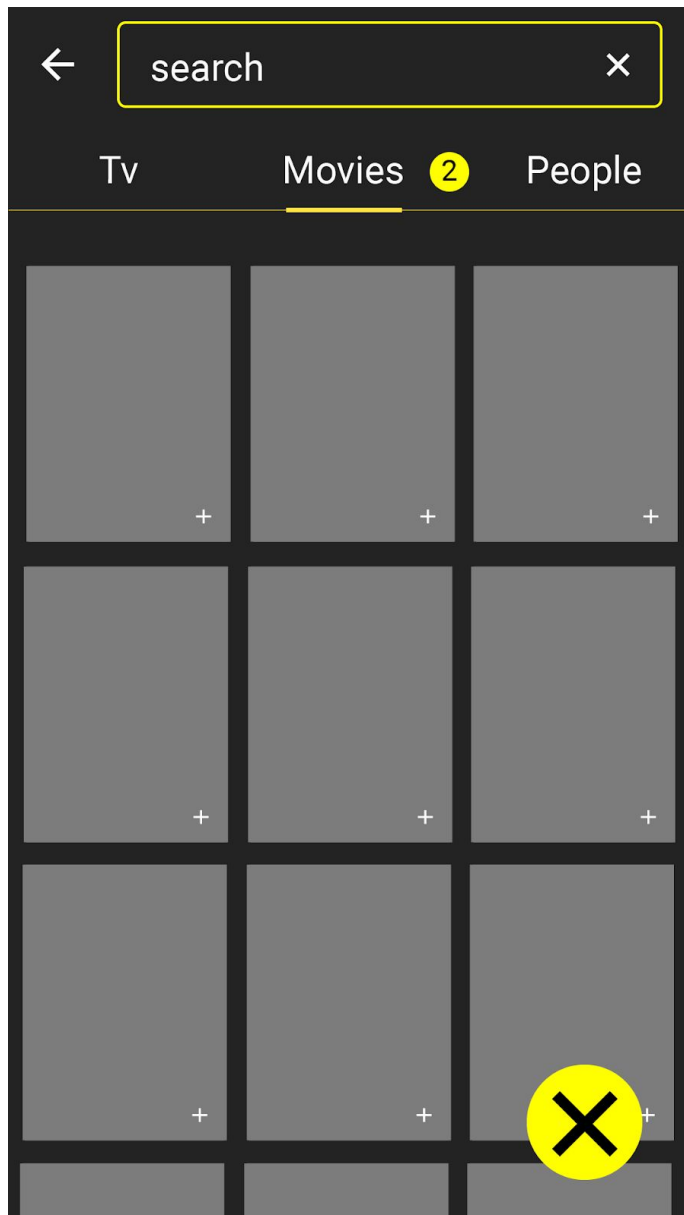
The second panel displays preset searches that may interest the user including popular, top rated and highly anticipated media.

The search results are shown below the second panel.

A floating action button is present to switch the screen from the current “preset” search mode to the manual search mode.

The “+” symbols on the results helps adding items quickly to lists without going to the details screen.

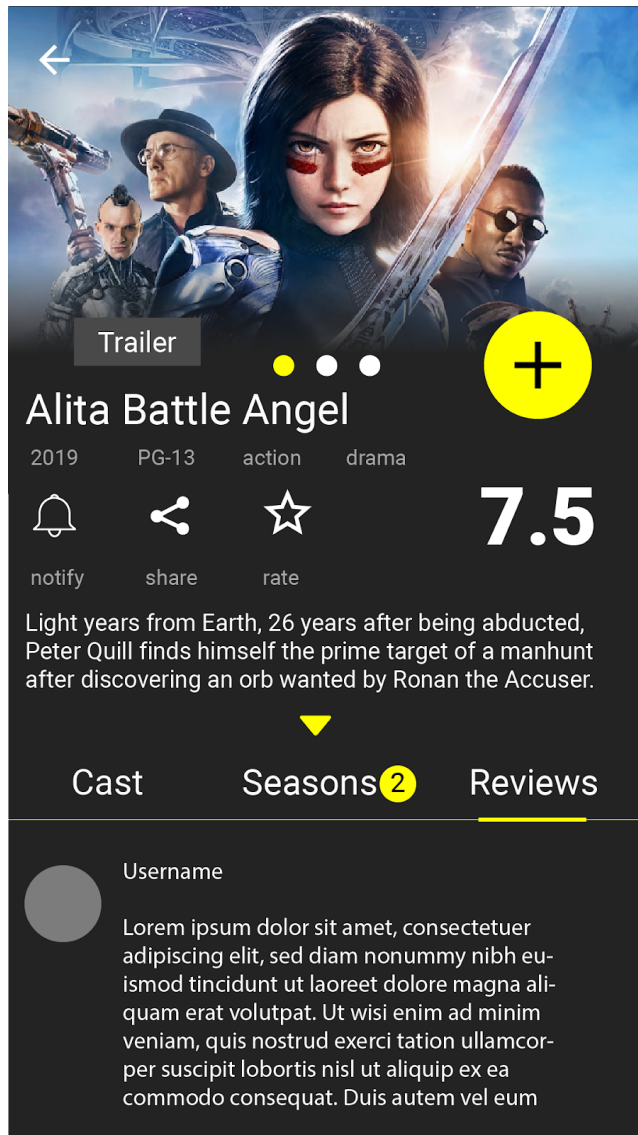
Search Screen Manual



This screen is accessed by pressing the floating action button from the preset search mode. A search bar is revealed for the user to find specific content.

The user can return back to the preset search mode by pressing the floating action button once again.

Screen 2

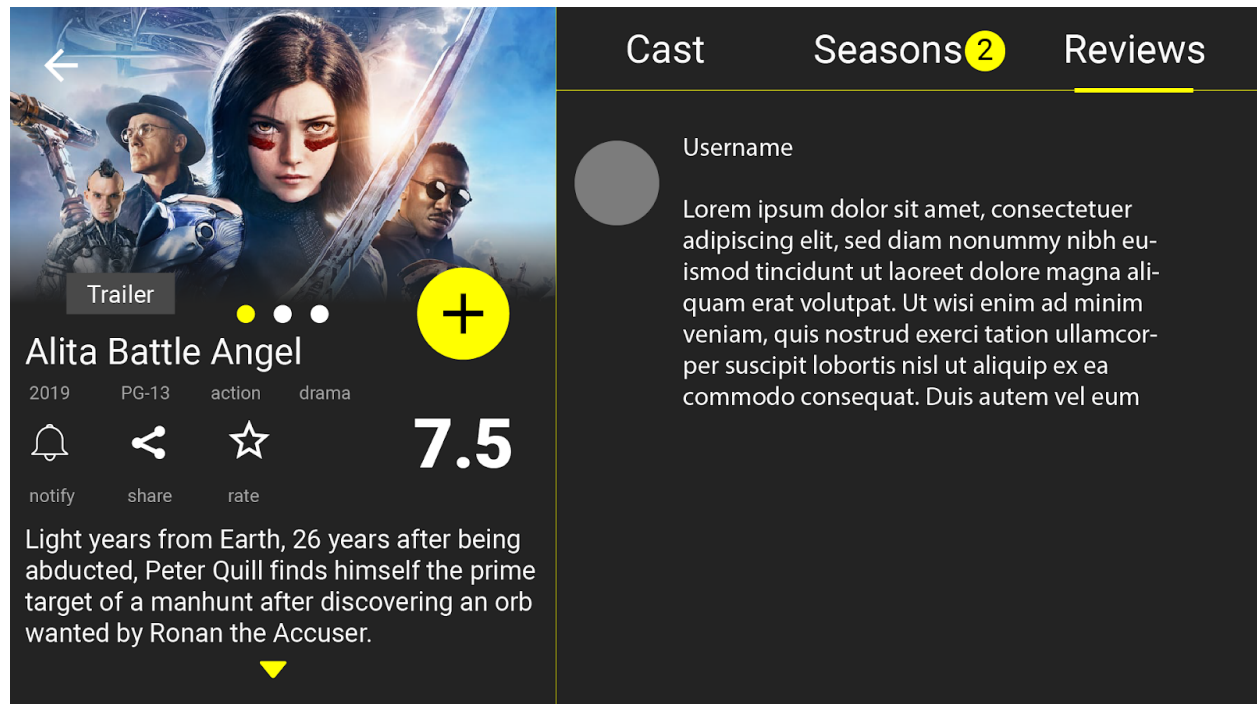


The details screen is accessed by touching any movie poster.

The screen provides the following functionality:

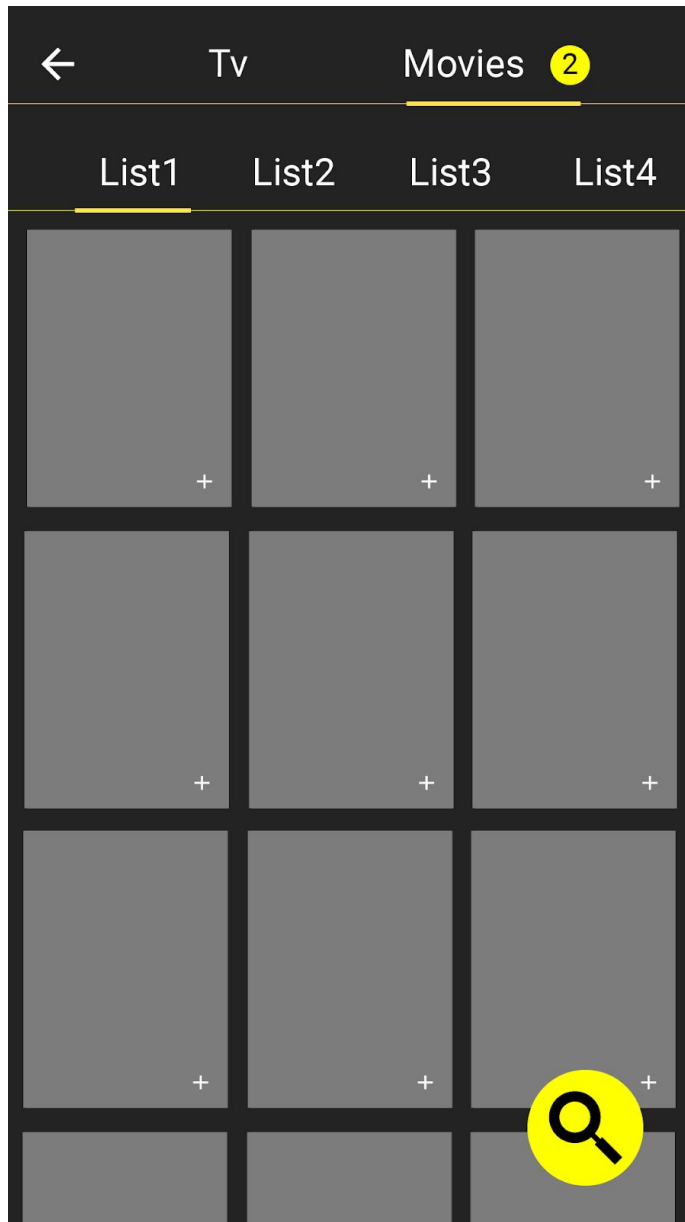
- Viewing trailers of the current media
- Adding the media to an existing list
- Notifying the user of future events related to the media including:
 - New episodes
 - dvd/ blu-ray releases
 - release on a streaming platform
- Sharing the media through android's share functionality
- Rating and reviewing the current media

Details Screen Phone Landscape



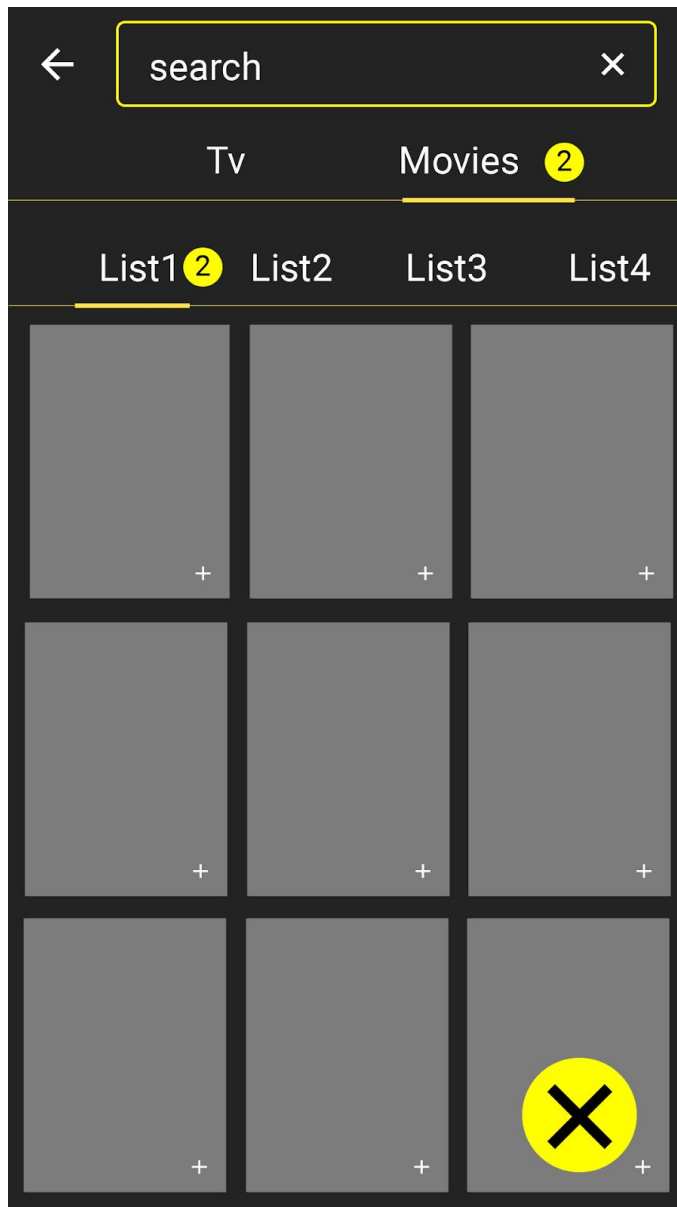
Landscape mode for the details screen to use horizontal space efficiently.

Lists Screen



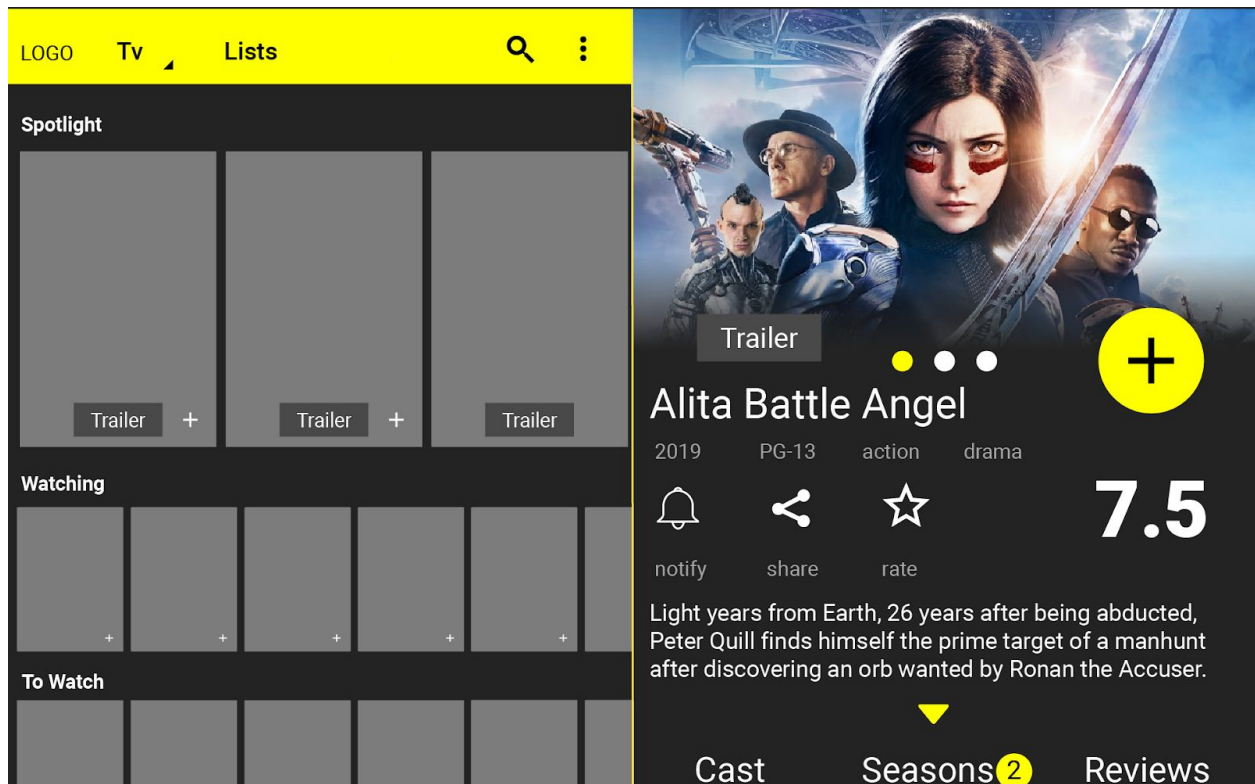
This screen shows all of the user's lists, both preprovided and self defined by the user. The user can easily navigate between Tv show lists and movie lists

Lists Screen (search mode)



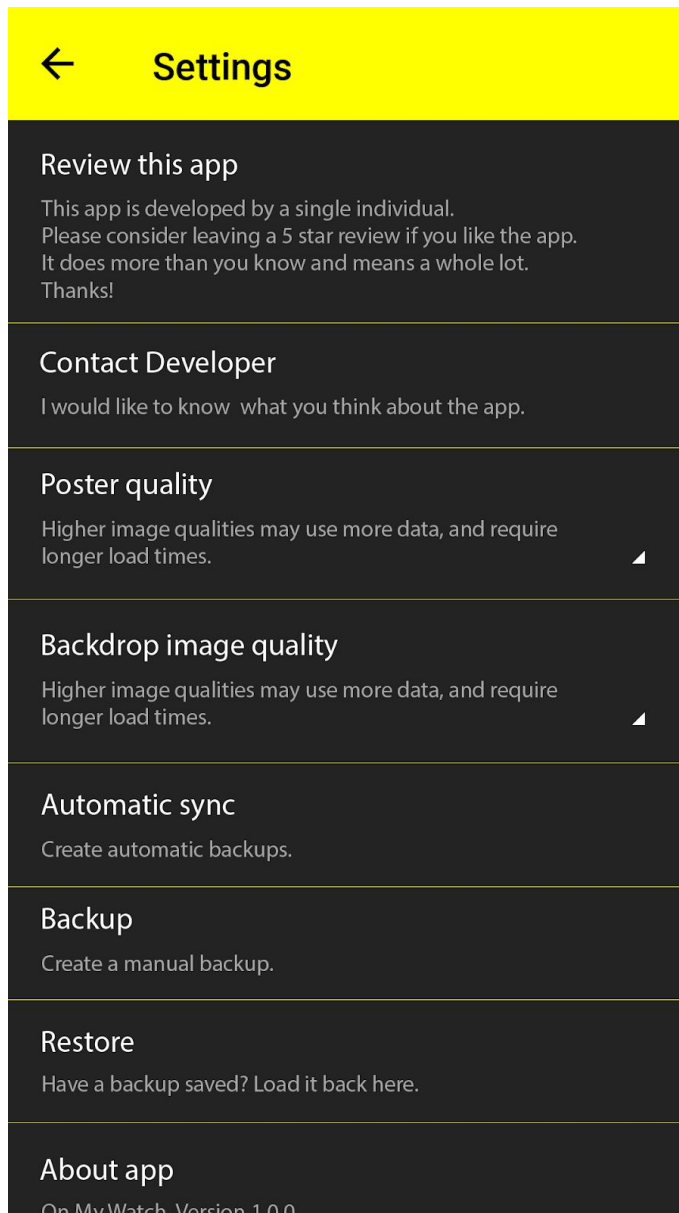
Search mode allows the user to quickly search their lists for specific items previously added.

Tablet Screen Landscape



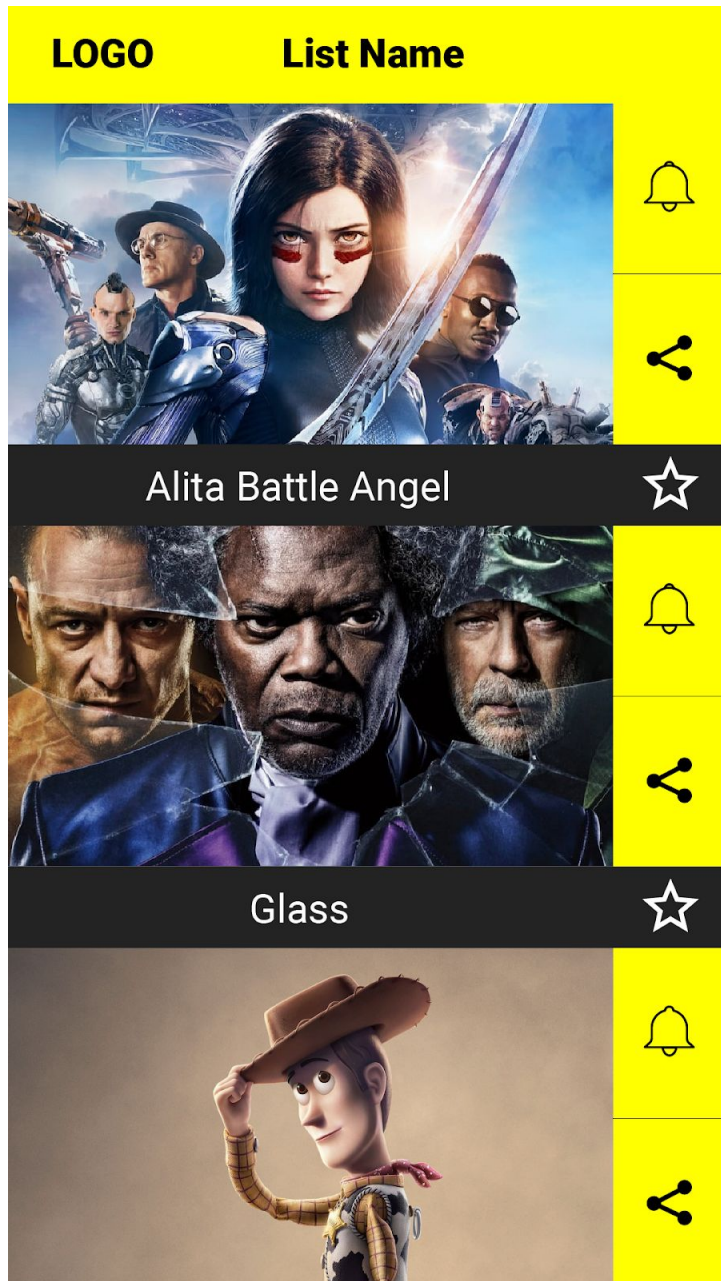
Tablet UI will be handled using a master - detail paradigm. The detail screen is persistent.

Settings Screen



The settings screen displays user defined preferences, houses settings for manual and automatic data backup, a way to contact the developer and version information about the app

Widget Screen



The widget displays the user's preselected list on the home screen with quick shortcuts to notify, share and rate media.

Key Considerations

Languages used:

The App will be written solely in the Java language

Resource handling:

All Strings and other resource types will be placed in their corresponding resource XML files

RTL support:

Specify that all the strings will be placed in strings.xml and your app will handle App will support RTL languages

Accessibility:

App will implement accessibility features to aid ease of use

Background services:

- App will perform all non UI task in a separate thread
- JobScheduler will be used for complex notification handling

Data persistence:

Data persistence will be handled through Android's Room Database and Shared Preferences and Google Drive.

Edge cases:

Problem foreseen with API. Since current API only supports 20 results per page. It becomes inconvenient for larger devices as every little scroll will result in waiting for page loads. A multi query library will be made. The library will run queries asynchronously and results will be stitched back in their proper order.

Used libraries:

StackPunch (self made recycler view library)

Android design library (28.0.0): material design implementation

Admob with Firebase (18.0.0): for generating revenue of free tier of the app

Firebase Core (17.0.0): for Firebase Analytics deep user insights

Butterknife (10.1.0): there will be lots of views to bind, butterknife will make this cleaner

Parceler (1.1.12): data can be passed through activities as whole objects instead of piecemeal

Gson (2.8.5): easy json parsing

Glide (4.9.0): easy image processing

Timber (4.7.1): easy logging helper
Espresso (3.1.0): for rigorous testing

- Runner (1.1.0)
- Rules (1.1.0)

FastAndroidNetworking (1.0.2): simplifying network queries

Describe how you will implement Google Play Services or other external services.

Ad mob will be used in a free version of the app to generate revenue and or incentivize purchasing the paid version of the app. Firebase Analytics will be used for deep user insights and bug reports.

Next Steps: Required Tasks

Task 1: Project Setup

Initial project configurations

- Configure libraries
- Configure gradle wrapper
- Configure SDK parameters

Task 2: Implement UI for Each Activity and Fragment

Create UI

- Build UI for Master Activity
- Build UI for Home Fragment
- Build UI for Tv Details Fragment
- Build UI for Movie Details Fragment
- Build UI for Landscape Tv Details Fragment
- Build UI for Landscape Movie Details Fragment
- Build UI for Search Fragment
- Build UI for Lists Fragment

Task 3: Create Java Classes

Create core UI display classes

- Create complementary UI Activity / Fragment classes
- Create View Adapters

Task 4: Connect to Api

Implement internet connectivity

- Handle api key safely
- Add uses internet permission in manifest
- Create Networking Class
- Create Movie Item Parser
- Create Tv Item Parser
- Create Season/Episode Item Parser

Task 5: Create RecyclerView Multi Query Library

Create library to query multiple API pages at a time since current API only supports 20 results per page. This is very inconvenient for larger devices as every little scroll will result in waiting for page loads

Task 6: Handle data persistence

Create local database

- Create lists database using Room

Task 7: Notifications and background services

Handle notifications, background services and Firebase Analytics

Task 8: Branch free and paid versions of app

Handle implementation of Firebase Admob in free version

Task 9: Create Settings

Create UI tests to ensure proper code functionality:

- Create pro version purchasable in app
- Create option to choose image quality
- Create Google Drive backup functionality
- Create contact developer functionality
- Create review functionality

Task 10: Create UI Tests

- Create UI tests to ensure proper code functionality:
- Add “About App” information

Task 11: Add UI Animations

Add life to the UI by implementing animation:

- Create transition start/stop animations
- Create Icon animations / morphing
- Add ripple/selector animations for touch targets

Task 12: Create build variant

Create Build Variant:

- Configure automatic testing
 - Configure automatic app signing
-